

1. 环境要求

1.1 软件依赖

软件	版本要求	验证命令
JDK	≥ 17	java -version
Maven	≥ 3.6	mvn -v
Node.js	≥ 18	node -v
npm	≥ 8	npm -v
MySQL	≥ 8.0	mysql --version

1.2 端口占用

服务	端口	说明
后端 API	8080	Spring Boot
前端页面	3000	Vite Dev Server
MySQL	3306	数据库

若端口被占用，需修改配置文件后重新启动。

2. 部署步骤

2.1 获取项目

项目位于 D:\Learn\JAVAee\Java01，目录结构：

Java01/

├── init.sql # 数据库初始化脚本
├── meeting-api/ # 后端项目
├── meeting-web/ # 前端项目
└── docs/ # 文档

2.2 初始化数据库

登录 MySQL，执行初始化脚本

mysql -u root -p --default-character-set=utf8mb4 < D:\Learn\JAVAee\Java01\init.sql

执行成功后创建以下表：

- user (含默认管理员 admin/admin123)
- meeting
- registration

2.3 配置数据库连接

编辑 meeting-api/src/main/resources/application.yml：

spring:

datasource:

url:

jdbc:mysql://localhost:3306/meeting_db?useUnicode=true&characterEncoding=utf-

```
8&serverTimezone=Asia/Shanghai
  username: root
  password: your_password_here    # ← 改为你的 MySQL 密码
```

2.4 启动后端

```
cd D:\Learn\JAVAAee\Java01\meeting-api
```

Linux/Mac

```
mvn spring-boot:run
```

Windows PowerShell

```
mvn.cmd spring-boot:run
```

启动成功日志：

```
Tomcat started on port 8080 (http)
```

```
Started MeetingApplication in 1.873 seconds
```

2.5 安装前端依赖

```
cd D:\Learn\JAVAAee\Java01\meeting-web
```

```
npm install
```

2.6 启动前端

```
npm run dev
```

启动成功输出：

```
VITE v5.x.x  ready in xxx ms
```

```
→ Local:   http://localhost:3000/
```

2.7 访问系统

打开浏览器访问：<http://localhost:3000>

3. 配置说明

3.1 后端配置 (application.yml)

server:

```
  port: 8080                                # 后端服务端口
```

spring:

datasource:

```
    url: jdbc:mysql://...    # 数据库连接 URL
    username: root           # 数据库用户名
    password: your_password  # 数据库密码
```

mybatis:

configuration:

```
    log-impl: ...            # SQL 日志（开发用 StdOutImpl，生产可关闭）
```

3.2 前端配置 (vite.config.js)

server: {

```
  port: 3000,                  # 前端开发服务器端口
```

```
  proxy: {
```

```
    '/api': {
```

```
        target: 'http://localhost:8080', # 后端 API 地址
        changeOrigin: true
    }
}
}
```

生产环境需用 Nginx 等反向代理替换 Vite Proxy。

4. 生产部署

4.1 后端打包

```
cd meeting-api
```

```
mvn package -Dmaven.test.skip=true
```

生成 target/meeting-api-1.0.0.jar, 运行:

```
java -jar target/meeting-api-1.0.0.jar
```

或指定外部配置:

```
java -jar target/meeting-api-1.0.0.jar --spring.config.location=file:./application.yml
```

4.2 前端构建

```
cd meeting-web
```

```
npm run build
```

生成 dist/ 目录, 部署到 Nginx:

```
server {
    listen 80;
    server_name your-domain.com;

    root /path/to/meeting-web/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

5. 验证部署

5.1 后端健康检查

```
curl http://localhost:8080/api/meeting/list
```

预期返回:

```
{"code": 200, "message": "success", "data": [...]}
```

5.2 前端访问

- 打开 `http://localhost:3000` → 应显示登录页面

- 使用 admin / admin123 登录 → 应进入会议列表

5.3 运行 API 测试

Postman 导入 meeting-api-test.postman_collection.json 运行集合测试。

6. 常见问题

Q1: 后端启动报端口被占用

查找 8080 端口进程

```
netstat -ano | findstr :8080
```

结束进程 (PID 替换为实际值)

```
taskkill /PID 1234 /F
```

Q2: 数据库连接失败

- 检查 MySQL 服务是否运行
- 检查 application.yml 中的用户名密码
- 确认 meeting_db 数据库已创建

Q3: 前端请求后端 404

- 确认后端已启动
- 检查 Vite proxy 配置中的 target 地址
- 浏览器访问 <http://localhost:8080/api/meeting/list> 直接测试

Q4: 前端页面空白

- 检查浏览器控制台有无报错
- 确认 npm install 成功执行
- 确认后端已启动 (前端依赖于 API 获取数据)